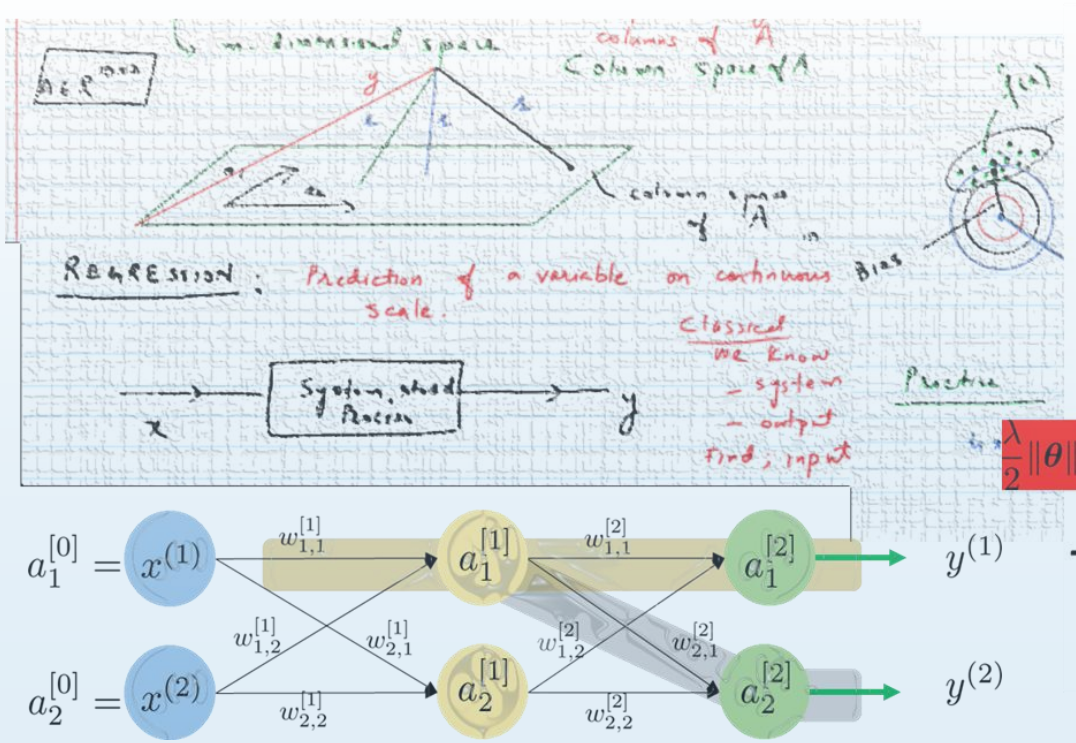
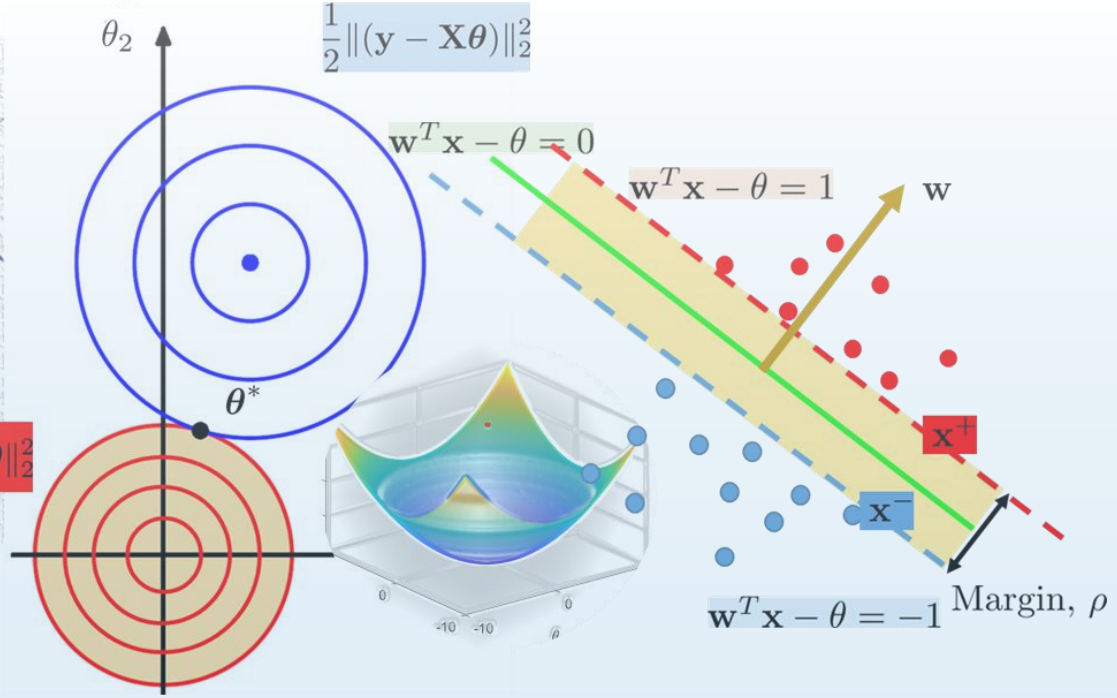




AI-501 Mathematics for AI

Machine Learning – Supervised Learning and kNN

Zubair Khalid
School of Science and Engineering

LUMS
A Not-for-Profit University

Supervised Learning Setup

Formulation

Using the adopted notation, we can formalize the supervised machine learning setup. We represent the entire training data as

$$D = \{(\mathbf{x_1}, y_1), (\mathbf{x_2}, y_2), \ldots, (\mathbf{x_n}, y_n)\} \subseteq \mathcal{X}^d \times \mathcal{Y}$$

Here $\mathcal{X}^d$ - d dimensional feature space and $\mathcal{Y}$ is the label space.

Regression: $\mathcal{Y} = \mathbf{R}$ (prediction on continuous scale)

Classification: $\mathcal{Y} = \{0, 1\}$ or $\mathcal{Y} = \{-1, 1\}$ or $\mathcal{Y} = \{1, 2\}$ (Binary classification)

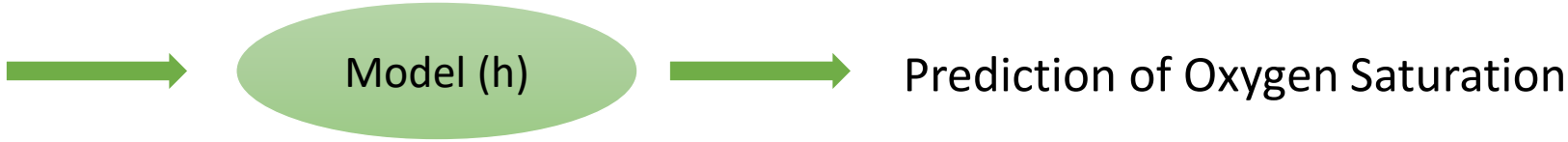
$\mathcal{Y} = \{1, 2, \ldots, M\}$ (M-class classification)

Supervised Learning Setup

Example

Data of 200 Patients:

- Age of the patient
- Cholesterol levels
- Glucose levels
- BMI
- Height
- Heart Rate
- Calories intake
- No. of steps taken



Supervised Learning Setup

Learning

Recall a problem in hand. We want to develop a model that can predict the label for the input for which label is unknown.

We assume that the data points $(\mathbf{x_i}, y_i)$ are drawn from some (unknown) distribution $P(X, Y)$.

Our goal is to learn the machine (model, function or hypothesis) h such that for a new pair $(\mathbf{x}, y)$ P, we can use h to obtin

$$h(\mathbf{x}) = y$$

with high probability or

$$h(\mathbf{x}) \approx y$$

in some optimal sense.

Supervised Learning Setup

Hypothesis Class

We call the set of possible functions or candidate models (linear model, neural network, decision tree, etc.) "the hypothesis class".

Denoted by $\mathcal{H}$

For a given problem, we wish to select hypothesis (machine) $h \in \mathcal{H}$.

Q: How?

A: Define hypothesis class $\mathcal{H}$ for a given learning problem.

Evaluate the performance of each candidate function and choose the best one.

Supervised Learning Setup

Q: How do we evaluate the performance?

A: Define a loss function to quantify the accuracy of the prediction.

Loss Function

Loss function should quantify the error in predicting y using hypothesis function h and input $\mathbf{x}$.

Denoted by $\mathcal{L}$.

Supervised Learning Setup

0/1 Loss Function:

Zero-one loss is defined as $\mathcal{L}_{0/1}(h) = \frac{1}{n}\sum_{i=1}^{n} 1 - \delta_{h(\mathbf{x_i})-y_i}$

Here $\delta_{h(\mathbf{x_i})-y_i}$ is the delta function defined as

$$\delta_k = \begin{cases} 1, & k = 0 \\ 0 & \text{otherwise} \end{cases}$$

Interpretation:

- Note normalization by the number of samples. This makes it the loss per sample.
- Loss function counts the number of mistakes made by hypothesis function on D.
- **Not used frequently due to non-differentiability and non-continuity.**

Supervised Learning Setup

Squared Loss Function:

Squared loss is defined as (also referred to as **mean-square error, MSE**)

$$\mathcal{L}_{\text{sq}}(h) = \frac{1}{n}\sum_{i=1}^{n}\left(h(\mathbf{x_i}) - y_i\right)^2$$

Interpretation:

- Again note normalization by the number of samples.
- Loss grows quadratically with the absolute error amount in each sample.

Root Mean Squared Error (RMSE):

RMSE is just square root of squared loss function:

$$\mathcal{L}_{\text{rms}}(h) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(h(\mathbf{x_i}) - y_i\right)^2}$$

Supervised Learning Setup

Absolute Loss Function (Mean Absolute Error):

Absolute loss is defined as

$$\mathcal{L}_{\text{abs}}(h) = \frac{1}{n}\sum_{i=1}^{n} |h(\mathbf{x_i}) - y_i|$$

Interpretation:

- Loss grows linearly with the absolute of the error in each prediction.
- Used in regression and suited for noisy data.

* All of the losses are non-negative

Supervised Learning Setup

Learning

We wish to select hypothesis (machine) $h \in \mathcal{H}$ such that

$$h^* = \min_{h \in \mathcal{H}} \mathcal{L}(h)$$

(Optimization problem)

Recall We assume that the data points $(\mathbf{x_i}, y_i)$ are drawn from some (unknown) distribution $P(X, Y)$.

We can come up with a function h after solving this minimization problem that gives low loss on our data.

Q: How can we ensure that hypothesis *h* will give low loss on the input not in *D*?

Supervised Learning Setup

To illustrate this, let us consider a model h trained on every input in D, that is, giving zero loss. Such function is referred to as memorizer and can be formulated as follows

$$h(\mathbf{x}) = \begin{cases} y_i, & \exists\ (\mathbf{x}_i, y_i) \in D, \quad \mathbf{x}_i = \mathbf{x}, \\ 0, & \text{otherwise} \end{cases}$$

Interpretation:

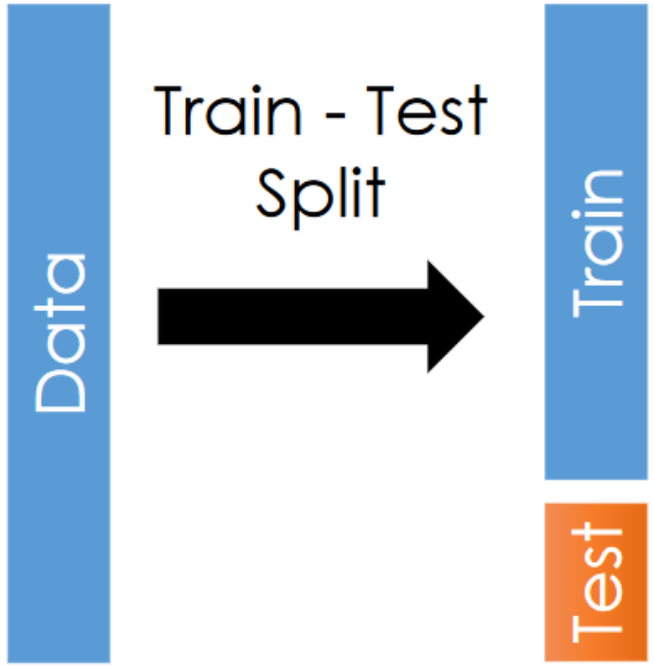
- 0% loss error on the training data (Model is fit to every data point in D).
- Large error for some input not in D
- First glimpse of ***overfitting.***

Revisit:

Q: How can we ensure that hypothesis *h* will give low loss on the input not in *D*?

A: Train/Test Split

Supervised Learning Setup



Generalization: The Train-Test Split

To resolve the overfitting issue, we usually split D into train and test subsets:

- D_{TR} as the training data, (70, 80 or 90%)
- D_{TE} as the test data, (30, 20, or 10%)

How to carry out splitting?

- Split should be capturing the variations in the distribution.
- Usually, we carry out splitting using i.i.d. sampling and time series with respect to time

You can only use the test dataset once after deciding on the model using training dataset

Supervised Learning Setup

Learning (Revisit after train-test split)

We had the following optimiztion problem as

$$h^* = \min_{h \in \mathcal{H}} \mathcal{L}(h)$$

We generalize it as

$$h^* = \min_{h \in \mathcal{H}} \frac{1}{|D_{\text{TR}}|} \sum_{(\mathbf{x}, y) \in D_{\text{TR}}} \mathcal{L}(\mathbf{x}, y)|h)$$

Evaluation

Loss on the testing data is given by

$$\epsilon_{\text{TE}} = \frac{1}{|D_{\text{TE}}|} \sum_{(\mathbf{x}, y) \in D_{\text{TE}}} \mathcal{L}(\mathbf{x}, y)|h*)$$

Supervised Learning Setup

Generalization: The Train-Test Split

At times, we usually split D into three subsets, that is, the training data is further divided into traaining and validation datasets:

- D_{TR} as the training data, (80%)
- D_{VA} as the validation data, (10%)
- D_{TE} as the test data, (10%)

Q: Idea:

Validation data is used to evaluate the loss for a function h that is determined using the learning on the training data-set. If the loss on validation data is high for a given h, the hypothesis or model needs to be changed.

Supervised Learning Setup

Generalization: The Train-Test Split

More explanation* to better understand the difference between validation and test data:

- **Training set:** A set of examples used for learning, that is to fit the parameters of the hypothesis (model).

- **Validation set:** A set of examples used to tune the hyper-parameters of the hypothesis function, for example to choose the number of hidden units in a neural network OR the order of polynomial approximating the data.

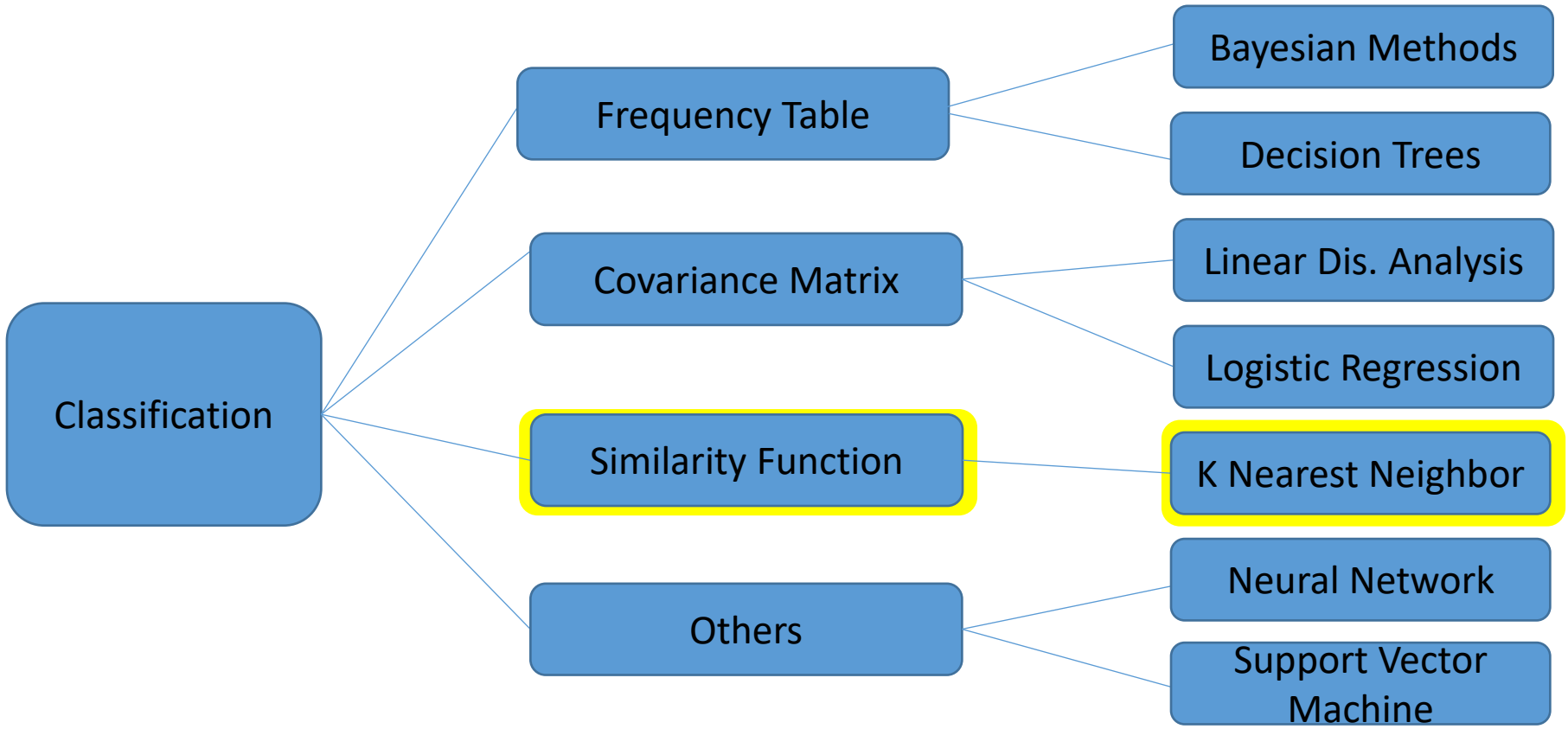
- **Test set:** A set of examples **used** only to assess the performance of a fully-specified model or hypothesis.

Adapted from *Brian Ripley, Pattern Recognition and Neural Networks, 1996

Supervised Learning

Classification Algorithms or Methods

Predicting a categorical output is called classification

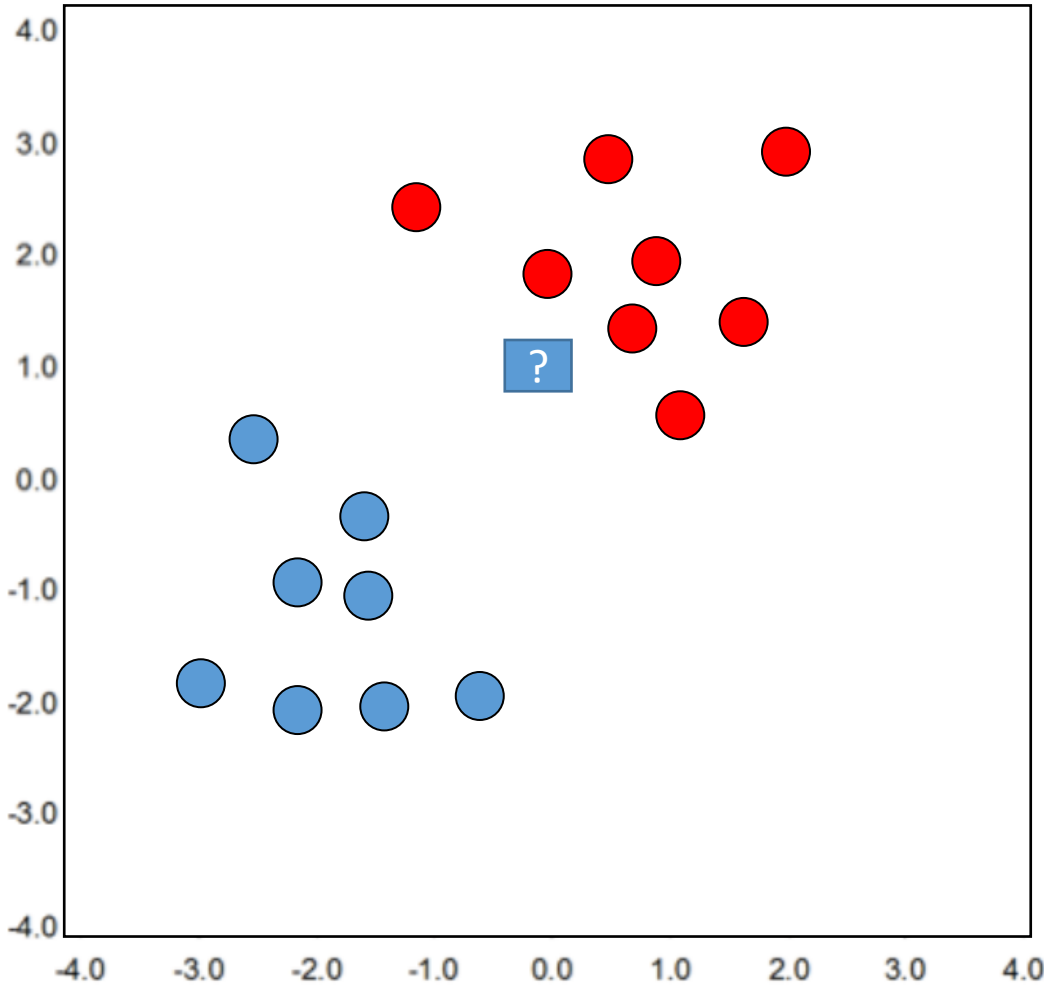


Outline

- k-Nearest Neighbor (kNN) Algrorithm Overview
- Algorithm Formulation
- Choice of k
- Storage, Time Complexity Analysis

k-Nearest Neighbor (kNN) Algorithm

Idea:



- Two classes, two features
- We want to assign label to unknown data point?
- Label should be red.

k-Nearest Neighbor (kNN) Algorithm

Idea:

- We have similar labels for similar features.
- We classify new test point using similar training data points.

Algorithm overview:

- Given some new test point x for which we need to predict the class y.
- Find **most similar** data-points in the training data.
- Classify x "like" these **most** similar data points.

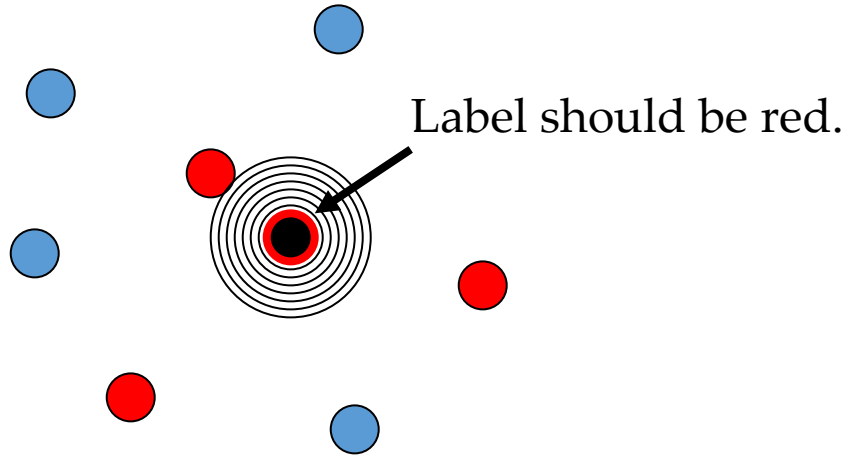
Questions:

- How do we determine the similarity?
- How many similar training data points to consider?
- How to resolve inconsistencies among the training data points?

k-Nearest Neighbor (kNN) Algorithm

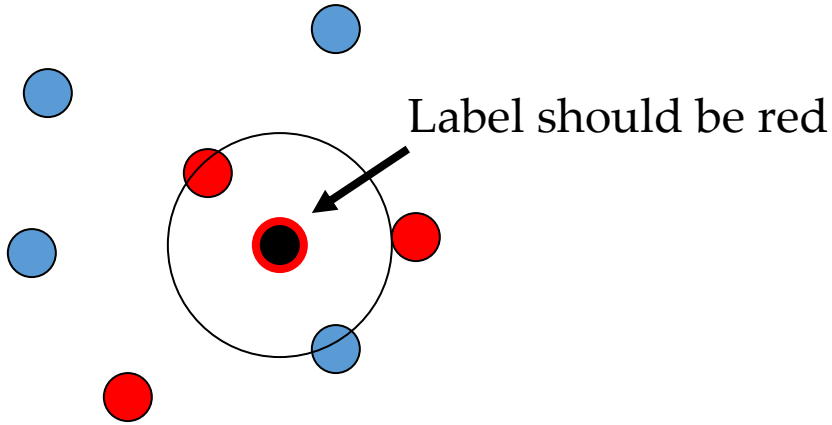
1-Nearest Neighbor:

Simplest ML Classifier
*Idea: Use the label of the **closest** known point*

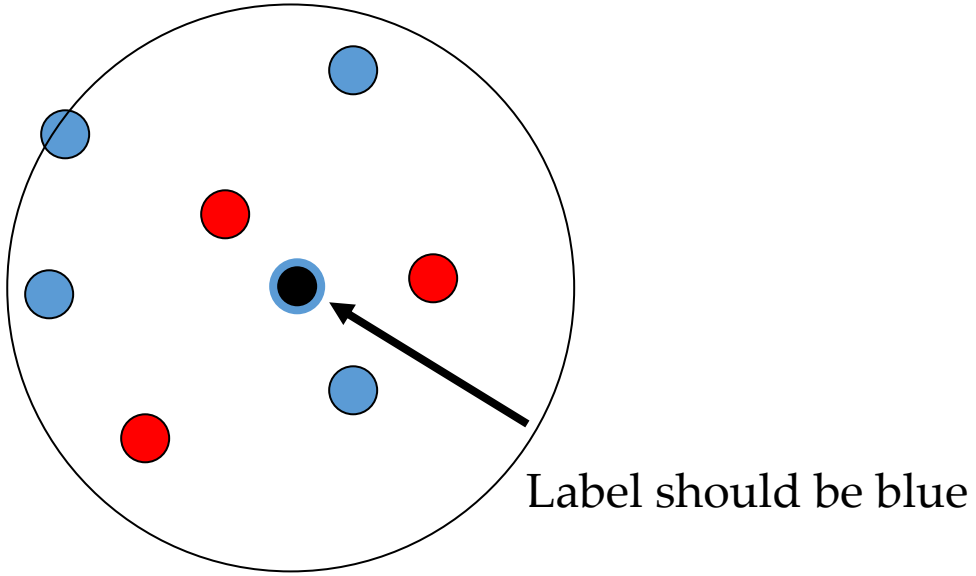


Generalization:

*Determine the label of **k** nearest neighbors and assign the most frequent label*



k=3



k=7

k-Nearest Neighbor (kNN) Algorithm

Formal Definition:

We have a training dataset D containing n points:

$$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

- x_i represents the features of a data point.
- y_i represents the class label.
- We have M possible classes: $y_i \in \{1, 2, \ldots, M\}$.

- **Query Point:** For a new point x, we want to find its k nearest neighbors.
- **Neighbor Set (S_x):** S_x is a subset of D containing the k points that are closest to x.

k-Nearest Neighbor (kNN) Algorithm

Formal Definition:

We look at the labels of the points in the neighbor set S_x and take a majority vote (mode).

$$h(x) = \text{Majority Vote of labels in } S_x$$

- **Instance-based learning:** The algorithm does not build a model during training. It simply stores the training data.
- **Flexibility:** It easily adapts to unseen data by calculating distances to the new point x when a prediction is needed.

k-Nearest Neighbor (kNN) Algorithm

Characteristics of kNN:

- No assumptions about the distribution of the data
- Non-parametric algorithm
  - No parameters

- Hyper-Parameters
  - k (number of neighbors)
  - Distance metric (to quantify similarity)

k-Nearest Neighbor (kNN) Algorithm

Characteristics of kNN:

- Complexity (both time and storage) of prediction increases with the size of training data. We will review this shortly.

- Can also be used for regression (average or inverse distance weighted average)
  - For example,

$$y = \frac{1}{k}\sum_{i=1}^{k} y_i, \quad (\mathbf{x}_i, y_i) \in S_{\mathbf{x}}$$

k-Nearest Neighbor (kNN) Algorithm

Practical issues:

- For binary classification problem, use odd value of k. Why?

- In case of a tie:
  - Use prior information
  - Use 1-nn classifier or k-1 classifier to decide

- Missing values in the data
  - Average value of the feature.

k-Nearest Neighbor (kNN) Algorithm

Distance Metric:

- For categorical vaiable, use Hamming Distance

$$\text{dist}(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{d} 1 - \delta_{x_i - x_i'}$$

k-Nearest Neighbor (kNN) Algorithm

Practical issues in computing distance:

- Mismatch in the values of data
  - Issue: Distance metric is mapping from d-dimensional space to a scaler. The values should be of the same order along each dimension.

  - Solution: Data Normalization

Outline

- k-Nearest Neighbor (kNN) Algroithm Overview
- Algorithm Formulation
- **Choice of k**
- Storage, Time Complexity Analysis

k-Nearest Neighbor (kNN) Algorithm

Choice of k:

- k=1

Sensitive to noise

High variance

Increasing k makes algorithm less sensitive to noise

- k=n

Decreasing k enables capturing finer structure of space

Idea: Pick k not too large, but not too small (depends on data)

How?

k-Nearest Neighbor (kNN) Algorithm

Choice of k:

- Learn the best hyper-parameter, k using the data.
- Split data into training and validation.
- Start from k=1 and keep iterating by carrying out (5 or 10, for example) cross-validation and computing the loss on the validation data using the training data.
- Choose the value for k that minimizes validation loss.
- This is the only learning required for kNN.

Outline

- k-Nearest Neighbor (kNN) Algroithm Overview
- Algorithm Formulation
- Choice of k
- **Storage, Time Complexity Analysis**

k-Nearest Neighbor (kNN) Algorithm

Algorithm:

Steps:

Computational Complexity

1. *Find distance between given test point and feature vector of every point in D.*

   Noting n number of data points we have and each feature vector $\mathbf{x}$ is d-dimensional. $\mathcal{O}(dn)$

2. *Find k points in D closest to the given test point vector to form a set S_X.*

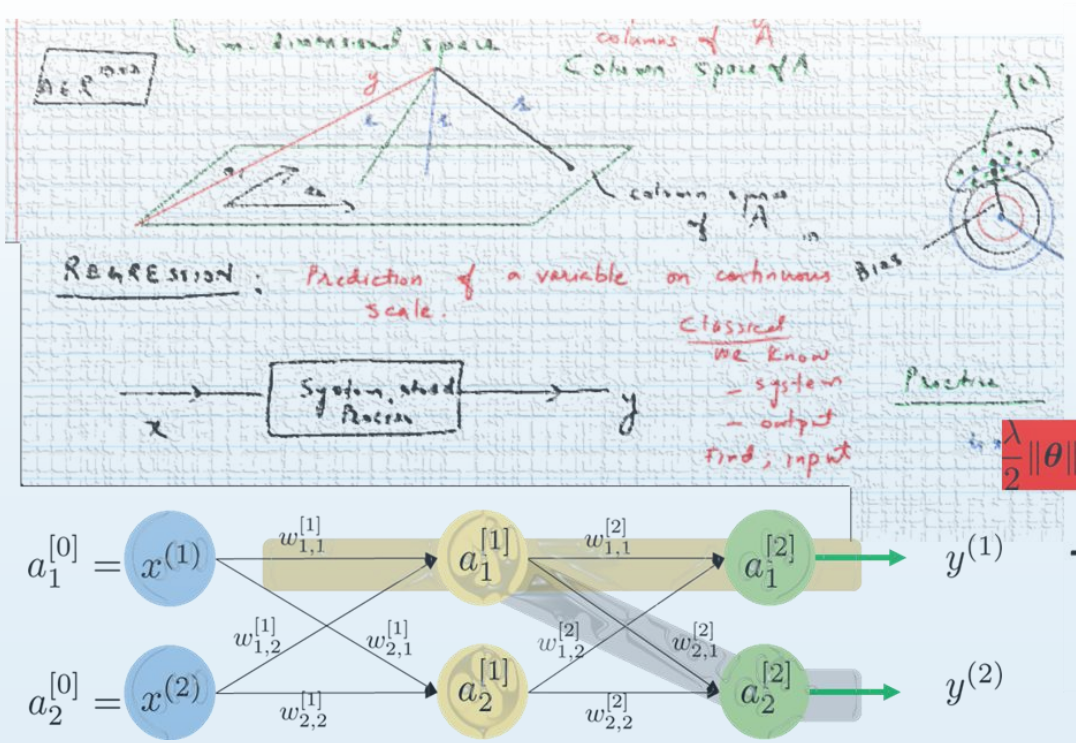
   Finding k-th smallest distance using median of medians method. $\mathcal{O}(n)$

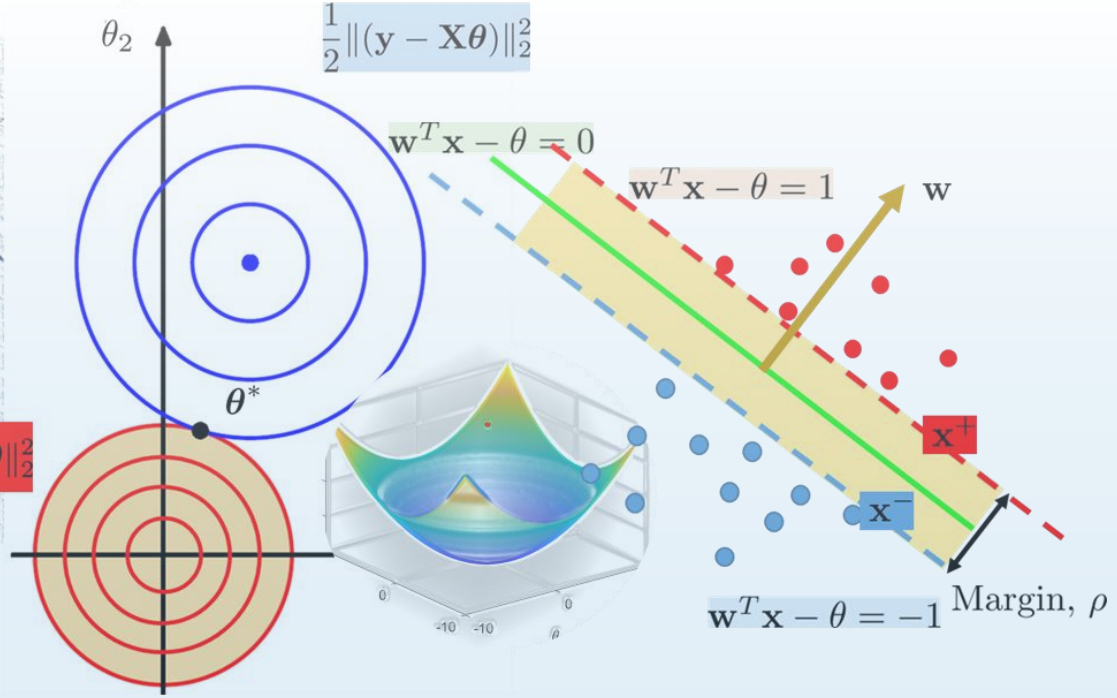
   Finding k data-points in D with distance less than the k-th smallest distance $\mathcal{O}(n)$

3. *Find the most frequent label in the set S_x and assign it to the test point.* $\mathcal{O}(k)$

Computational Complexity: $\mathcal{O}(dn)$

Space Complexity: $\mathcal{O}(dn)$





AI-501 Mathematics for AI

Machine Learning – Classifier's Performance Evaluation

Zubair Khalid
School of Science and Engineering

LUMS
A Not-for-Profit University

Outline

- Classification Accuracy (0/1 Loss)
- TP, TN, FP and FN
- Confusion Matrix
- Sensitivity, Specificity, Precision Trade-offs, ROC, AUC
- F1-Score and Matthew's Correlation Coefficient

Evaluation of Classification Performance

Classification Accuracy, Misclassification Rate (0/1 Loss):

$$\mathcal{L}_{0/1}(h) = \frac{1}{n}\sum_{i=1}^{n} 1 - \delta_{h(\mathbf{x_i})-y_i}$$

$$\delta_k = \begin{cases} 1, & k = 0 \\ 0 & \text{otherwise} \end{cases}$$

- For each test-point, the loss is either 0 or 1; whether the prediction is correct or incorrect.
- Averaged over n data-points, this loss is a 'Misclassification Rate'.

Interpretation:

- Misclassification Rate: Estimate of the probability that a point is incorrectly classified.
- Accuracy = 1- Misclassification rate

Issue:

- Not meaningful when the classes are imbalanced or skewed.

Evaluation of Classification Performance

Classification Accuracy (0/1 Loss):

Example:

- Predict if a bowler will not bowl a **no-ball**?
  - Assuming 15 no-balls in an inning, a **model that says 'Yes' all the time** will have **95%** accuracy.
  - Using accuracy as performance metric, we can say that a model is very accurate, but it is not useful or valuable in fact.

Why?

- Total points: 315 (assuming other balls are legal ☺)
- No-ball label: Class 0 (4.76% are from this class)
- Not a no-ball label: Class 1 (95.24% are from this class)

Imbalanced Classes

Evaluation of Classification Performance

TP, TN, FP and FN:

- *Consider a binary classification problem.*

$$D = \{(\mathbf{x_1}, y_1), (\mathbf{x_2}, y_2), \ldots, (\mathbf{x_n}, y_n)\} \subseteq \mathcal{X}^d \times \mathcal{Y}$$

$\mathcal{Y} = \{0, 1\}$ (Referring 0 as Negative, 1 as Positive)

y - Actual labels, Ground truth, Gold labels or Standards

We have a classifier (hypothesis function) $h(\mathbf{x}) = \hat{y}$.

$y, \hat{y}$ - Positive (1) or Negative (0)

$\hat{y}$ - True if $\hat{y} = y$, False if $\hat{y} \neq y$

Evaluation of Classification Performance

TP, TN, FP and FN:

- TP - True Positive
  - Number of points with $y = 1$ and are classified as $\hat{y} = 1$
- TN - True Negative
  - Number of points with $y = 0$ and are classified as $\hat{y} = 0$
- FP - False Positive
  - Number of points with $y = 0$ and are classified as $\hat{y} = 1$
- FN - False Negative
  - Number of points with $y = 1$ and are classified as $\hat{y} = 0$

Evaluation of Classification Performance

TP, TN, FP and FN:

Example:

- Predict if a bowler will not bowl a **no-ball**?
  - 15 no-balls in an inning (Total balls: 315)
  - Bowl no-ball (Class 0), Bowl regular ball (Class 1)
  - Model(*) predicted 10 no-balls (8 correct predictions, 2 incorrect)

- TP - True Positive
- TN - True Negative
- FP - False Positive
- FN - False Negative

- TP - 298
- TN - 8
- FP - 7
- FN - 2

* Assume you have a model that has been observing the bowlers for the last 15 years and used these observations for learning.

Evaluation of Classification Performance

Confusion Matrix (Contingency Table):

- (TP; TN; FP; FN); usefully summarized in a table, referred to as confusion matrix:
  - the rows correspond to predicted class ($\hat{y}$)
  - and the columns to true class (y)

		Actual Labels		
		1 (Positive)	**0 (Negative)**	Total
Predicted Labels	**1 (Positive)**	**TP**	**FP**	Predicted Total Positives
	0 (Negative)	**FN**	**TN**	Predicted Total Negatives
	Total	P= TP+FN Actual Total Positives	N= P+TN Actual Total Negatives	

Evaluation of Classification Performance

Confusion Matrix:

Example:

- Disease Detection :

Given pathology reports and scans, predict heart disease

- Yes: 1, No: 0

		Actual Labels		
		1 (Positive)	0 (Negative)	Total
Predicted Labels	1 (Positive)	**TP = 100**	**FP = 10**	110
	0 (Negative)	**FN = 5**	**TN = 50**	55
	Total	P = 105	N = 60	

Interpretation:

Out of 165 cases

- Predicted: "Yes" 110 times, and "No" 55 times
- In reality: "Yes" 105 times, and "No" 60 times

Evaluation of Classification Performance

Confusion Matrix:

Example:

- Predict if a bowler will not bowl a **no-ball**?

		Actual Labels		
		1 (Positive)	**0 (Negative)**	Total
Predicted Labels	**1 (Positive)**	**TP = 298**	**FP = 7**	305
	0 (Negative)	**FN = 2**	**TN = 8**	10
	Total	P = 300	N = 15	

Interpretation:

Out of 315 balls, we had 15 no-balls.

- Model predicted 305 regular balls and 10 no-balls (8 correct predictions, 2 incorrect).

Evaluation of Classification Performance

Confusion Matrix:

		Actual Labels		
		1 (Positive)	0 (Negative)	Total
Predicted Labels	1 (Positive)	TP	FP	Predicted Total Positives
	0 (Negative)	FN	TN	Predicted Total Negatives
	Total	P= TP+FN Actual Total Positives	N= P+TN Actual Total Negatives	

Metrics using Confusion Matrix:

- **Accuracy**: Overall, how frequently is the classifier correct?

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total}} = \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$$

- **Misclassification or Error Rate**: Overall, how frequently is it wrong?

$$1 - \text{Accuracy} = \frac{\text{FP} + \text{FN}}{\text{Total}} = \frac{\text{FP} + \text{FN}}{\text{P} + \text{N}}$$

- **Sensitivity or Recall or True Positive Rate (TPR)**: How often does it predict Positive when it is actually Positive?

$$TPR = S_e = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}}$$

Evaluation of Classification Performance

Confusion Matrix:

		Actual Labels		
		1 (Positive)	0 (Negative)	Total
Predicted Labels	1 (Positive)	TP	FP	Predicted Total Positives
	0 (Negative)	FN	TN	Predicted Total Negatives
	Total	P= TP+FN Actual Total Positives	N= P+TN Actual Total Negatives	

Metrics using Confusion Matrix:

- **False Positive Rate:** Actual Negative, how often does it predict Positive?

$$FPR = \frac{\text{FP}}{\text{TN} + \text{FP}} = \frac{\text{FP}}{\text{N}}$$

- **Specificity or True Negative Rate (TNR):** When it's actually Negative, how often does it predict Negative?

$$TNR = S_p = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{\text{TN}}{\text{N}} = 1 - FPR$$

- **Precision:** When it predicts Positive, how often is it Positive?

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Evaluation of Classification Performance

Confusion Matrix Metrics:

		Actual Labels		
		1 (Positive)	0 (Negative)	Total
Predicted Labels	1 (Positive)	TP	FP	*Predicted Total Positives*
	0 (Negative)	FN	TN	*Predicted Total Negatives*
	Total	P= TP+FN *Actual Total Positives*	N= P+TN *Actual Total Negatives*	

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\frac{\text{TN}}{\text{TN} + \text{FN}}$$ Negative Predicted Value

$$TPR = S_e = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}}$$

$$TNR = S_p = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{\text{TN}}{\text{N}}$$

Evaluation of Classification Performance

Confusion Matrix:

	Actual Labels			
Predicted Labels		1 (Positive)	0 (Negative)	Total
	1 (Positive)	TP = 100	FP = 10	110
	0 (Negative)	FN = 5	TN = 50	55
	Total	P = 105	N = 60	

Metrics using Confusion Matrix (Example: Disease Prediction):

- **Accuracy**: *Disease/Healthy prediction accuracy*

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total}} = \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$$ **= (100+50)/165 = 0.91**

- ***Misclassification or Error Rate***: *Disease/Healthy misclassification rate*

$$1 - \text{Accuracy} = \frac{\text{FP} + \text{FN}}{\text{Total}} = \frac{\text{FP} + \text{FN}}{\text{P} + \text{N}}$$ **= (10+5)/165 = 0.09**

- ***Sensitivity or Recall or True Positive Rate (TPR)***: *When it's positive, how often does the model detected disease?*

$$TPR = S_e = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}}$$ **= 100/105 = 0.95**

Evaluation of Classification Performance

Confusion Matrix:

		Actual Labels		
		1 (Positive)	0 (Negative)	Total
Predicted Labels	1 (Positive)	TP = 100	FP = 10	110
	0 (Negative)	FN = 5	TN = 50	55
	Total	P = 105	N = 60	

Metrics using Confusion Matrix (Example: Disease Prediction):

- **False Positive Rate:** Actually heathy, how often does it predict yes?

$$FPR = \frac{\text{FP}}{\text{TN} + \text{FP}} = \frac{\text{FP}}{\text{N}}$$ **= 10/60 = 0.17**

- **Specificity or True Negative Rate (TNR):** When it's actually health, how often does it predict healthy?

$$TNR = S_p = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{\text{TN}}{\text{N}}$$ **= 50/60 = 0.83**

- **Precision:** When it predicts disease, how often is it correct?

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$ **= 100/110 = 0.91**

Evaluation of Classification Performance

Confusion Matrix:

	Actual Labels		
Predicted Labels		1 (Positive)	0 (Negative)
	1 (Positive)	TP	FP
	0 (Negative)	FN	TN

Metrics using Confusion Matrix:

When to use which?

- *Disease Detection: We do not want FN*

$$TPR = S_e = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}}$$

- *Fraud Detection: We do not want FP*

$$TNR = S_p = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{\text{TN}}{\text{N}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Outline

- Classification Accuracy (0/1 Loss)
- TP, TN, FP and FN
- Confusion Matrix
- Sensitivity, Specificity, Precision Trade-offs, ROC, AUC
- F1-Score and Matthew's Correlation Coefficient

Evaluation of Classification Performance

Confusion Matrix:

Precision and Sensitivity (Recall) Trade-off:

- Disease Detection:

Sensitivity or Recall

$$TPR = S_e = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}}$$

Precision

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- Recall or Sensitivity (S_e); how good we are at detecting diseased people.
- **Precision:** How many have been correctly diagnosed as unhealthy.
- If we have diagnosed everyone unhealthy, S_e=1 (diagnose all unhealthy people correctly) but **Precision may be low** (because TN=0 that increases the value of FP).

		Actual Labels	
		1 (Positive)	0 (Negative)
Predicted Labels	1 (Positive)	TP	FP
	0 (Negative)	FN	TN

- We want high **Precision** and high S_e (=1, Ideally).
- We should combine precision and sensitivity to evaluate the performance of classifier.
  - F1-Score

Evaluation of Classification Performance

Confusion Matrix:

Sensitivity and Specificity Trade-off:

- Disease Detection:

Sensitivity or Recall

$$TPR = S_e = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}}$$

Specificity

$$TNR = S_p = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{\text{TN}}{\text{N}}$$

- S_p and S_e; how good we are at detecting **healthy** and **diseased** people, respectively.

- If we have diagnosed everyone healthy, S_p=1 (diagnose all healthy people correctly) but S_e=0 (diagnose all unhealthy people incorrectly)

- **Ideally:** we want S_p= S_e= 1 (perfect sensitivity and specificity) but unrealistic.

Evaluation of Classification Performance

Confusion Matrix:

ROC Curve and AUC:

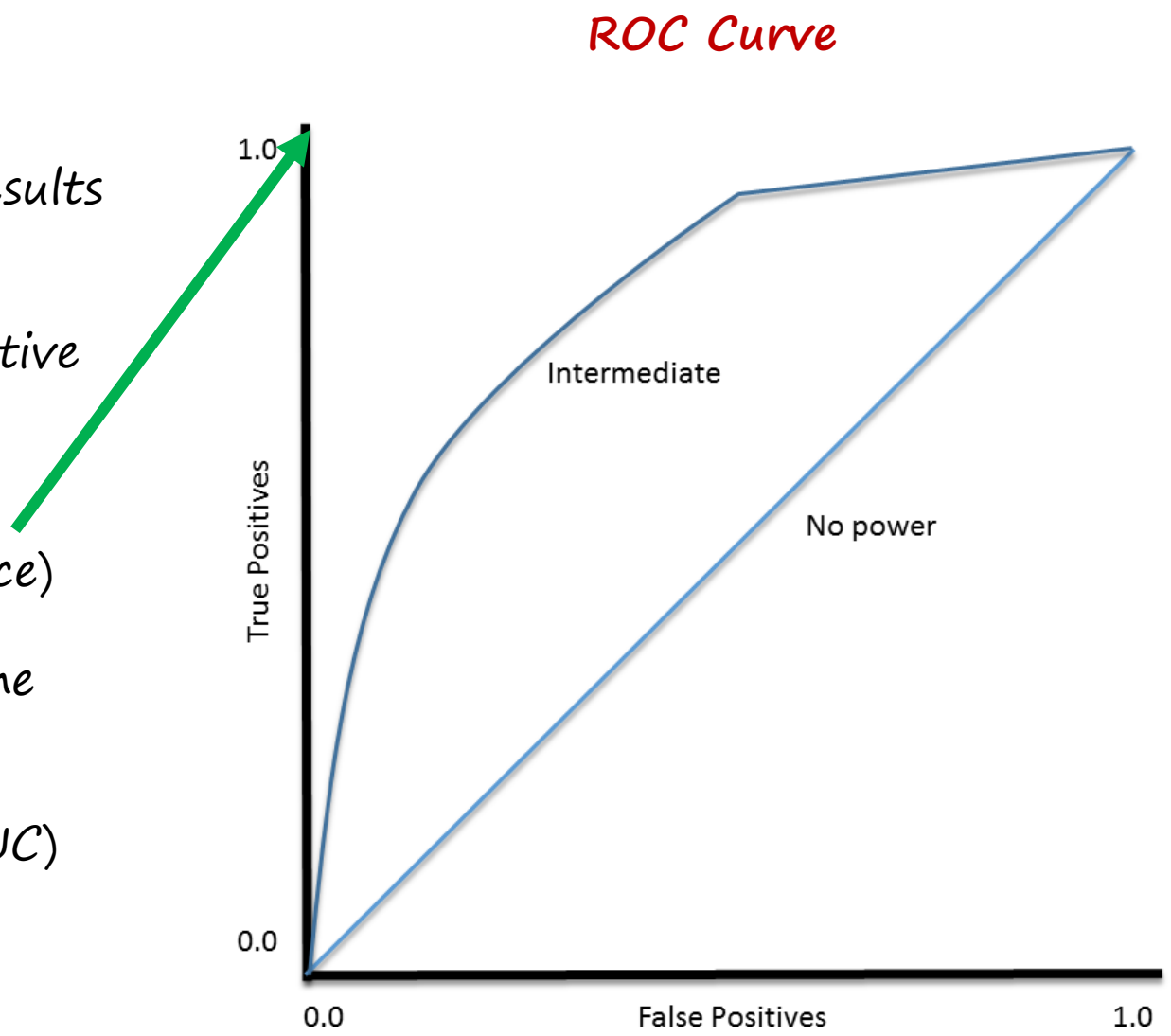
- TPR (Sensitivity): how many correct positive results occur among all positive samples.

- FPR (1 – Specificity): how many incorrect positive results occur among all negative samples.

- The best possible prediction method
  - $S_e = S_p = 1$ (Upper left corner of ROC space)

- Random guess; a point along a diagonal line (the so-called line of no-discrimination), No Power!

- Area Under the ROC Curve, abbreviated as (AUC) quantifies the power of the classifier.



Outline

- Classification Accuracy (0/1 Loss)
- TP, TN, FP and FN
- Confusion Matrix
- Sensitivity, Specificity, Precision Trade-offs, ROC, AUC
- F1-Score and Matthew's Correlation Coefficient

Evaluation of Classification Performance

F1-Score:

- We observed trade-off between recall and precision.

$$TPR = S_e = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}} \qquad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- Higher levels of recall may be obtained at the price of lower values of precision.

- We need to define a single measure that combines recall and precision or other metrics to evaluate the performance of a classifier.

- Some combined measures:
  - F1 Score
  - Matthew's Correlation Coefficient
  - 11-point average precision
  - The Breakeven point

Evaluation of Classification Performance

F1 Score:

- One measure that assesses recall and precision trade-off is weighted harmonic mean (HM) of recall and precision, that is,

$$F_\beta = \frac{1+\beta^2}{\frac{1}{\text{Precision}} + \frac{\beta^2}{\text{Recall}}}, \quad \beta \geq 0$$

For $\beta = 1$, we have harmonic mean of precision and recall, that is,

$$F_1 = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2(\text{Precision})(\text{Recall})}{(\text{Precision}) + (\text{Recall})} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

Evaluation of Classification Performance

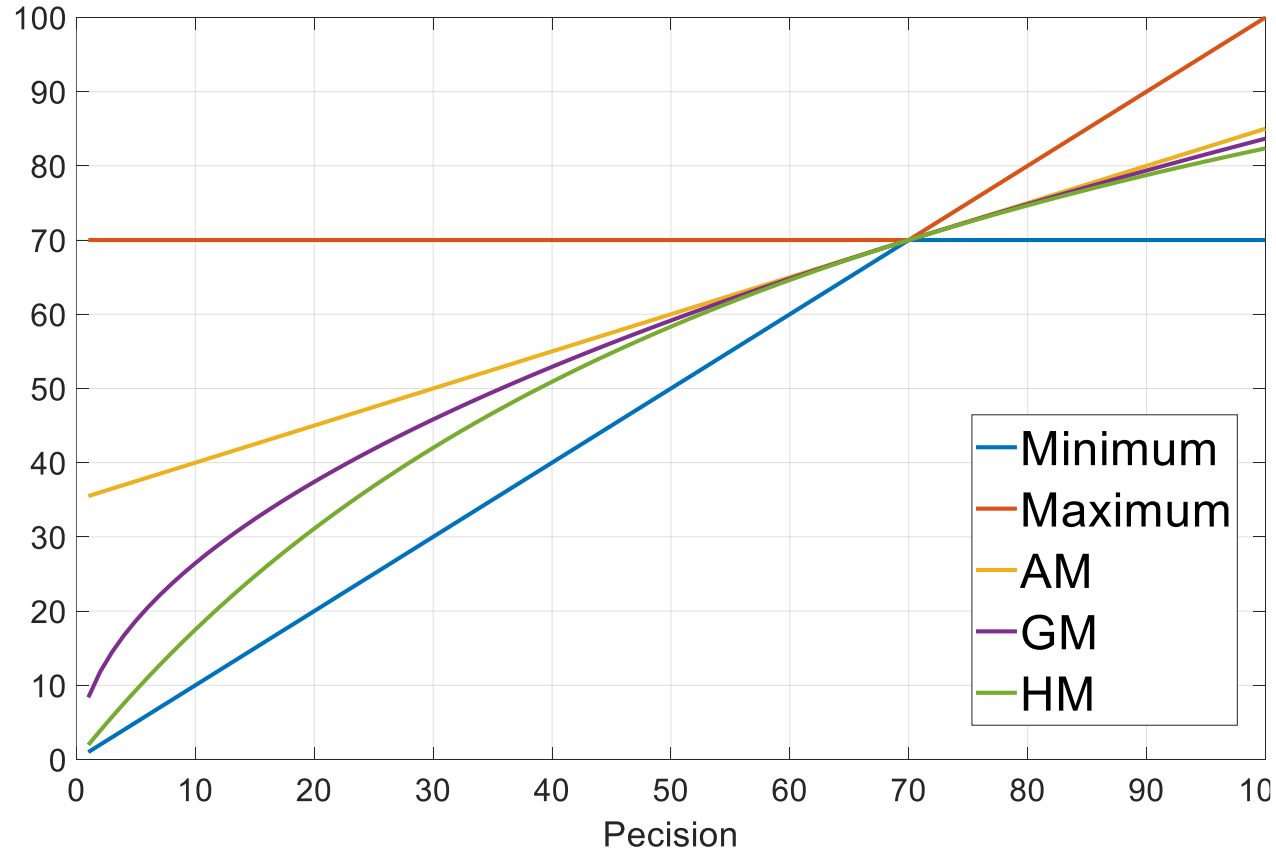
F1 Score:

Why harmonic mean?

- We could also use arithmetic mean (AM) or geometric mean (GM).
- HM is preferred as it penalizes model the most; a conservative average, that is, for two real positive numbers, we have

$$\mathrm{HM} \leq \mathrm{GM} \leq \mathrm{AM}$$

- Improvement in HM implies improvement in AM or GM.



Different means, minimum and maximum against precision. Recall=70% is fixed.